FirstForce — Project Evaluation

---

1. Introduction

Project title: ss-web — Web platform for capturing, OCR-processing and analysing medical fitness records (Fișa de aptitudine medicală) over MQTT

Team name: LF4A

Repository: https://github.com/Alexander1752/ss-web

Team members & roles:

Member	Primary role
Andrei Petrea	repo bootstrap, OCR service, CI/CD, Keycloak hardening
Alex Munteanu	Go API, React client, MQTT broker integration
Andrei Săcăluș	repository layer, photo/device routes, MinIO/S3 storage
Mihai-Lucian Pandelica	Backend / Keycloak integration — JWT validation, realm config, statistics
Flavius Mazilu	Security & compliance — SBOM pipeline, CodeQL, OSSF criticality score
Cristian-Alexandru Chiriac	Security — e.g. SECURITY.md , vulnerability triage, QA

2. Project Summary

2.1 Project overview

ss-web is a containerised web application that ingests photographed Romanian medical-fitness certificates ("Fișă de aptitudine medicală") from mobile and ESP32 devices over **MQTT (mTLS)**, runs **OCR** on them, parses the extracted text into structured medical records, persists them in **MongoDB**, and exposes a **React** dashboard for browsing, search, deletion and statistics. The platform is composed of:

- a React + TypeScript + Vite + Tailwind **frontend** behind NGINX, authenticated through **Keycloak** (OIDC, Authorization Code Flow + PKCE);
- a **Go** REST API that validates Keycloak-issued JWTs against the realm's JWKS, performs business logic and persists records in MongoDB;
- an **Eclipse Mosquitto** MQTT broker with mTLS-only listener on port 8883;
- a Python **OCR service** (Tesseract + Pillow) consuming images via MQTT and publishing extracted text back;
- **MinIO** object storage for image binaries and a **Postgres** instance backing Keycloak.

All services are orchestrated with docker compose . A regex-based medical-certificate parser (server/utils/medical_parser.go) extracts ~25 structured fields per record (personal data, control type, medical opinion, recommendations, next-exam date), enabling the *Statistics* page (control-type distribution, medical-opinion distribution, totals, periodic-check count).

2.2 Default OSSF Criticality Score

Computed with the official **ossf/criticality_score** CLI against the repo's original_pike.yml (the default Rob Pike weights), with the GitHub workflow at .github/workflows/criticality-score.yml automating the run on every push, every PR, and weekly.

> **Score:** 0.21426

---

3. Functionality, Documentation, Execution

3.1 Working features

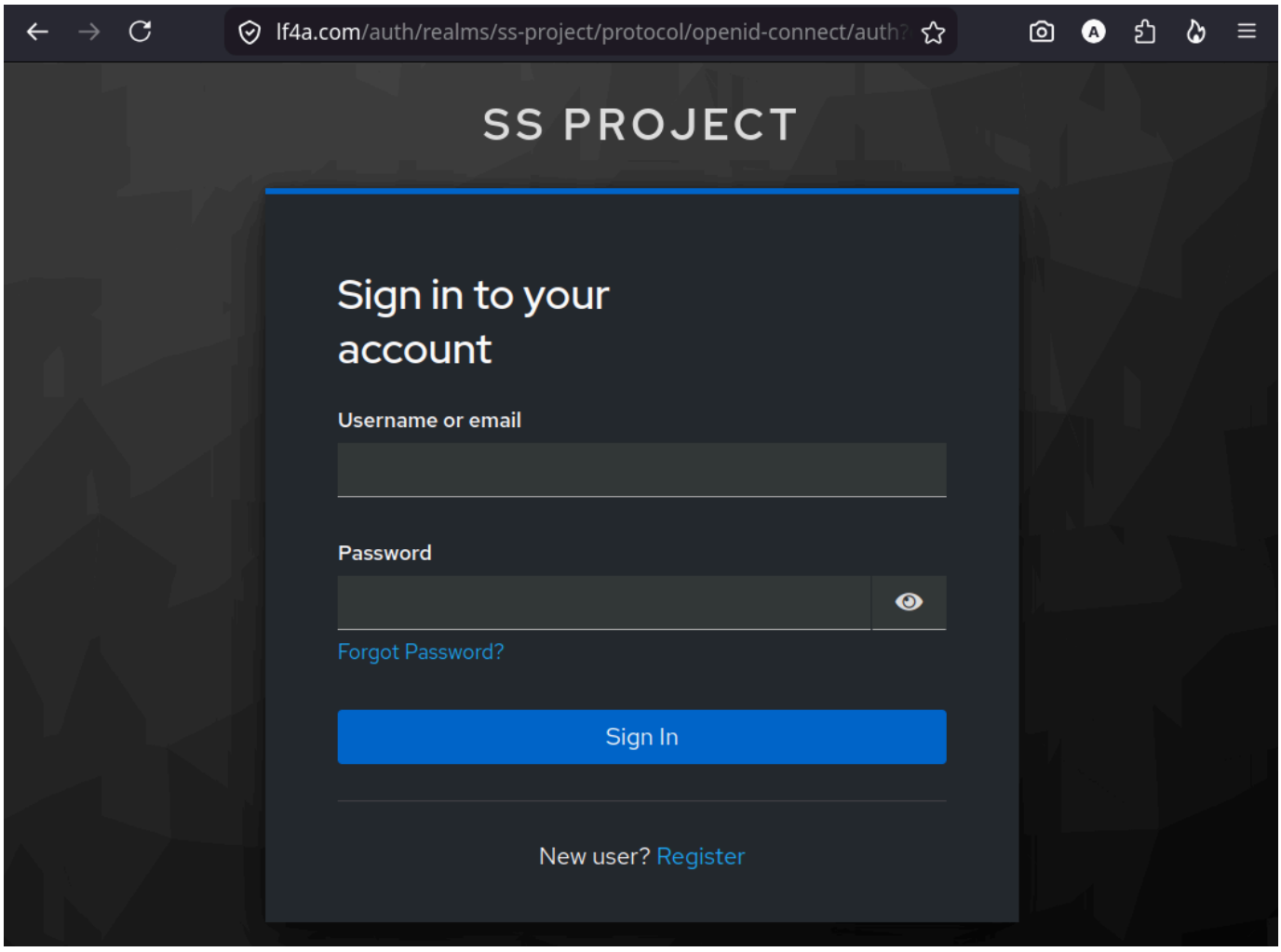
Feature	Implementation	Verified via
Image ingest over MQTT (mTLS)	server/main.go subscribes to ssproject/images/# ; broker requires client certs (broker/mosquitto.conf)	scripts/send_image.py , ESP-cam, Android client
OCR extraction	ocr/main.py subscribes to ssproject/ocr/requests , runs Tesseract (eng+ron), publishes back on ssproject/ocr/results	Photos page shows extracted text
Medical-certificate parsing	server/utils/medical_parser.go (regex + gap-analysis for OCR'd checkboxes)	server/utils/medical_parser_test.go
Photo gallery with search & filters	client/src/pages/photosPage/index.tsx , backend GET /photos?start=&end=&text=&device_id=	Photos page filter UI
Device management (Normal/Live)	client/src/pages/devicesPage/index.tsx , server/routes/device.go , MQTT device/id/#	Devices page; ESP camera Start/Stop Live
Statistics dashboard	client/src/pages/statisticsPage/index.tsx (Recharts bar/pie)	Statistics page (control-type + medical-opinion charts)
Keycloak authentication (OIDC + PKCE)	client/src/contexts/AuthContext.tsx (keycloak-js), server/routes/init.go::withAuth (JWKS validation)	Login redirect, JWT issued by auth.lf4a.com

Feature	Implementation	Verified via
Role-based access (admin vs user)	Frontend isAdmin , backend realm_access.roles claim check	Devices/admin actions guarded server-side
Single-photo & bulk deletion	DELETE /photos/{id} , DELETE /photos/all	Trash icon per card, Delete-All button
HTTPS for every public endpoint	NGINX → 443 for frontend, Go API listens on :5000 with ListenAndServeTLS , Keycloak 8443, Mongo-Express 8082	Browser padlock

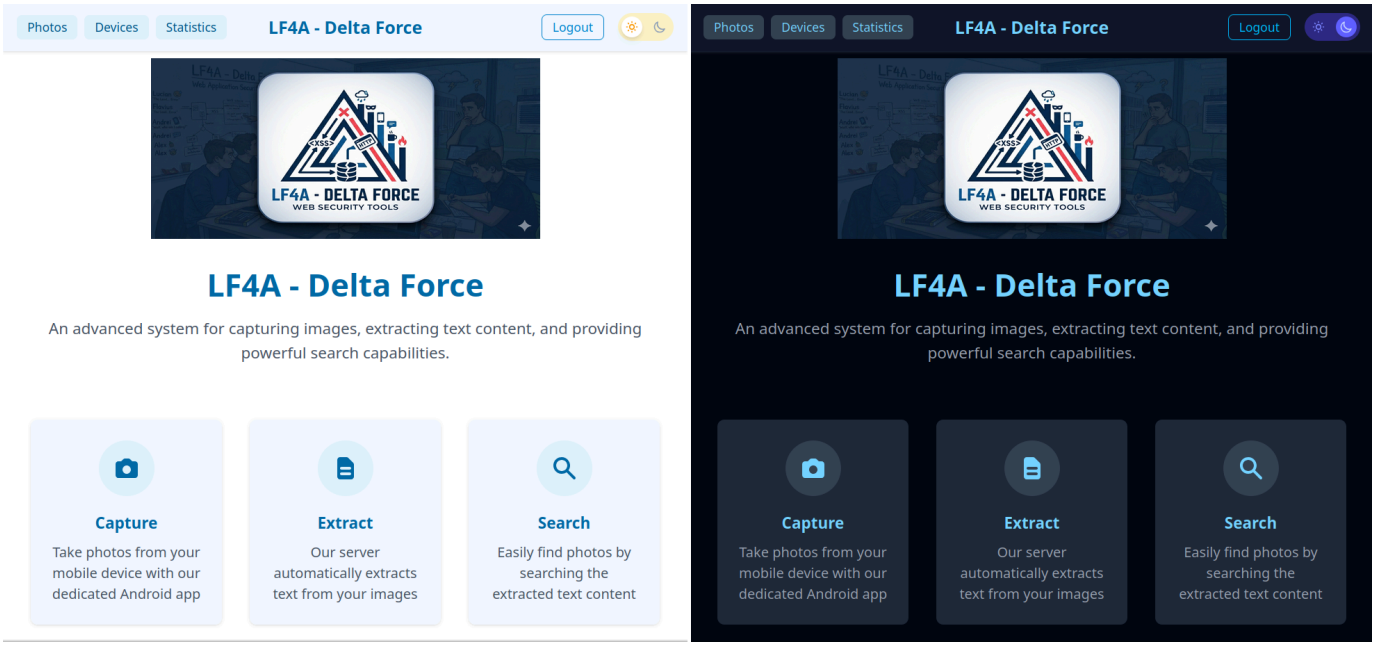
Example usage (golden path):

```
# 1. Start the stack
./start.sh                # or: docker compose up -d --build

# 2. Seed sample medical data (15 random records)
python3 scripts/seed_data.py

# 3. Send a real image through the MQTT pipeline
python3 scripts/send_image.py path/to/medical-cert.jpg

# 4. Browse the result
xdg-open https://localhost          # Photos / Statistics pages
```

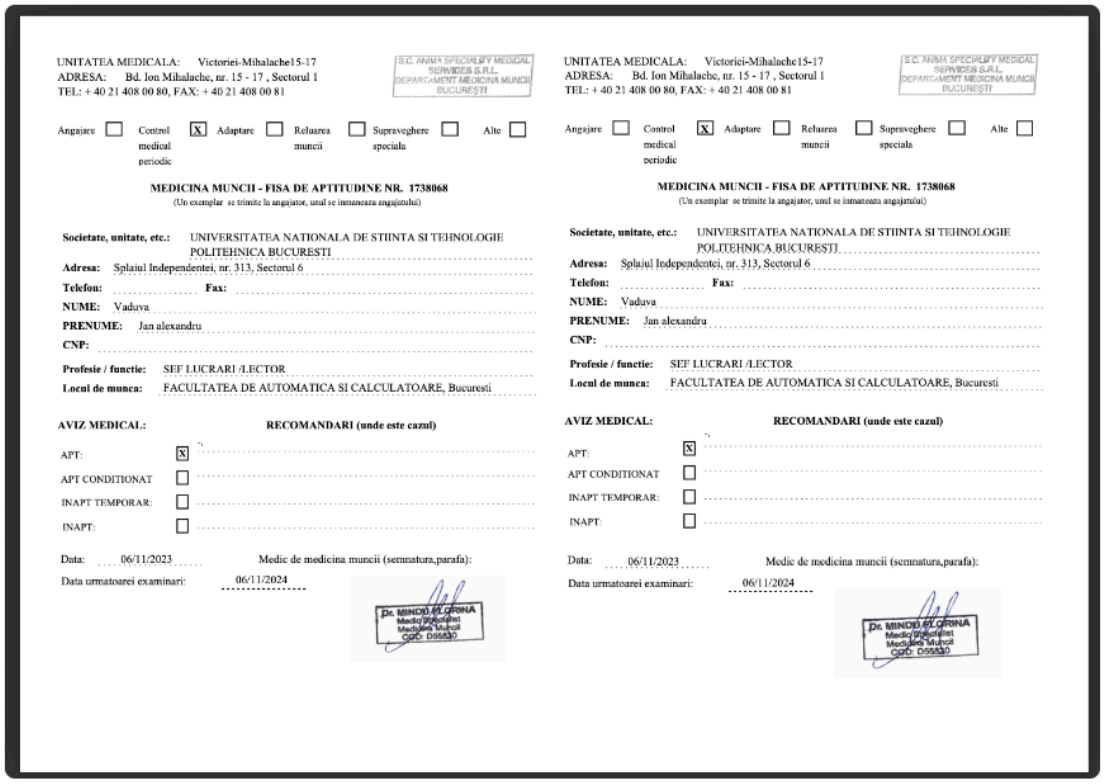
3.2 Application Screenshots



Login Page



Light VS Dark Mode



UNITATEA MEDICALA: Victoriei-Mihalache15-17
ADRESA: Bd. Ion Mihalache, nr. 15 - 17 , Sectorul 1
TEL: + 40 21 408 00 80, FAX: + 40 21 408 00 81

S.C. ANIMA SPECIALTY MEDICAL SERVICES S.R.L. DEPARTAMENT MEDICINA MUNCII BUCUREȘTI

Angajare ☐ Control medical periodic ☒ Adaptare ☐ Reluarea muncii ☐ Supraveghere speciala ☐ Alte ☐

MEDICINA MUNCII - FISA DE APTITUDINE NR. 1738068
(Un exemplar se trimite la angajator, unul se inmaneaza angajatului)

Societate, unitate, etc.: UNIVERSITATEA NATIONALA DE STIINTA SI TEHNOLOGIE POLITEHNICA BUCURESTI
Adresa: Splaiul Independentei, nr. 313, Sectorul 6
Telefon: **Fax:**
NUME: Vaduva
PRENUME: Jan alexandru
CNP:
Profesie / functie: SEF LUCRARI /LECTOR
Locul de munca: FACULTATEA DE AUTOMATICA SI CALCULATOARE, Bucuresti

AVIZ MEDICAL: **RECOMANDARI (unde este cazul)**

APT: ☒
APT CONDITIONAT ☐
INAPT TEMPORAR: ☐
INAPT: ☐

Data: 06/11/2023 Medic de medicina muncii (semnatura,parafa):
Data urmatoarei examinari: 06/11/2024

Dr. MINDRU ELORINA Medic Specialist Medicina Muncii Cod: D55830

UNITATEA MEDICALA: Victoriei-Mihalache15-17
ADRESA: Bd. Ion Mihalache, nr. 15 - 17 , Sectorul 1
TEL: + 40 21 408 00 80, FAX: + 40 21 408 00 81

S.C. ANIMA SPECIALTY MEDICAL SERVICES S.R.L. DEPARTAMENT MEDICINA MUNCII BUCUREȘTI

Angajare ☐ Control medical periodic ☒ Adaptare ☐ Reluarea muncii ☐ Supraveghere speciala ☐ Alte ☐

MEDICINA MUNCII - FISA DE APTITUDINE NR. 1738068
(Un exemplar se trimite la angajator, unul se inmaneaza angajatului)

Societate, unitate, etc.: UNIVERSITATEA NATIONALA DE STIINTA SI TEHNOLOGIE POLITEHNICA BUCURESTI
Adresa: Splaiul Independentei, nr. 313, Sectorul 6
Telefon: **Fax:**
NUME: Vaduva
PRENUME: Jan alexandru
CNP:
Profesie / functie: SEF LUCRARI /LECTOR
Locul de munca: FACULTATEA DE AUTOMATICA SI CALCULATOARE, Bucuresti

AVIZ MEDICAL: **RECOMANDARI (unde este cazul)**

APT: ☒
APT CONDITIONAT ☐
INAPT TEMPORAR: ☐
INAPT: ☐

Data: 06/11/2023 Medic de medicina muncii (semnatura,parafa):
Data urmatoarei examinari: 06/11/2024

Dr. MINDRU ELORINA Medic Specialist Medicina Muncii Cod: D55830

EXTRACTED TEXT

UNITATEA MEDICALA: __ Victoriei-Mihalache15-17
ADRESA: Bd. Ion Mihalache, nr. 15 - 17, Sectorul 1
TEL: + 40 21 408 00 80, FAX: + 40 21 408 00 81

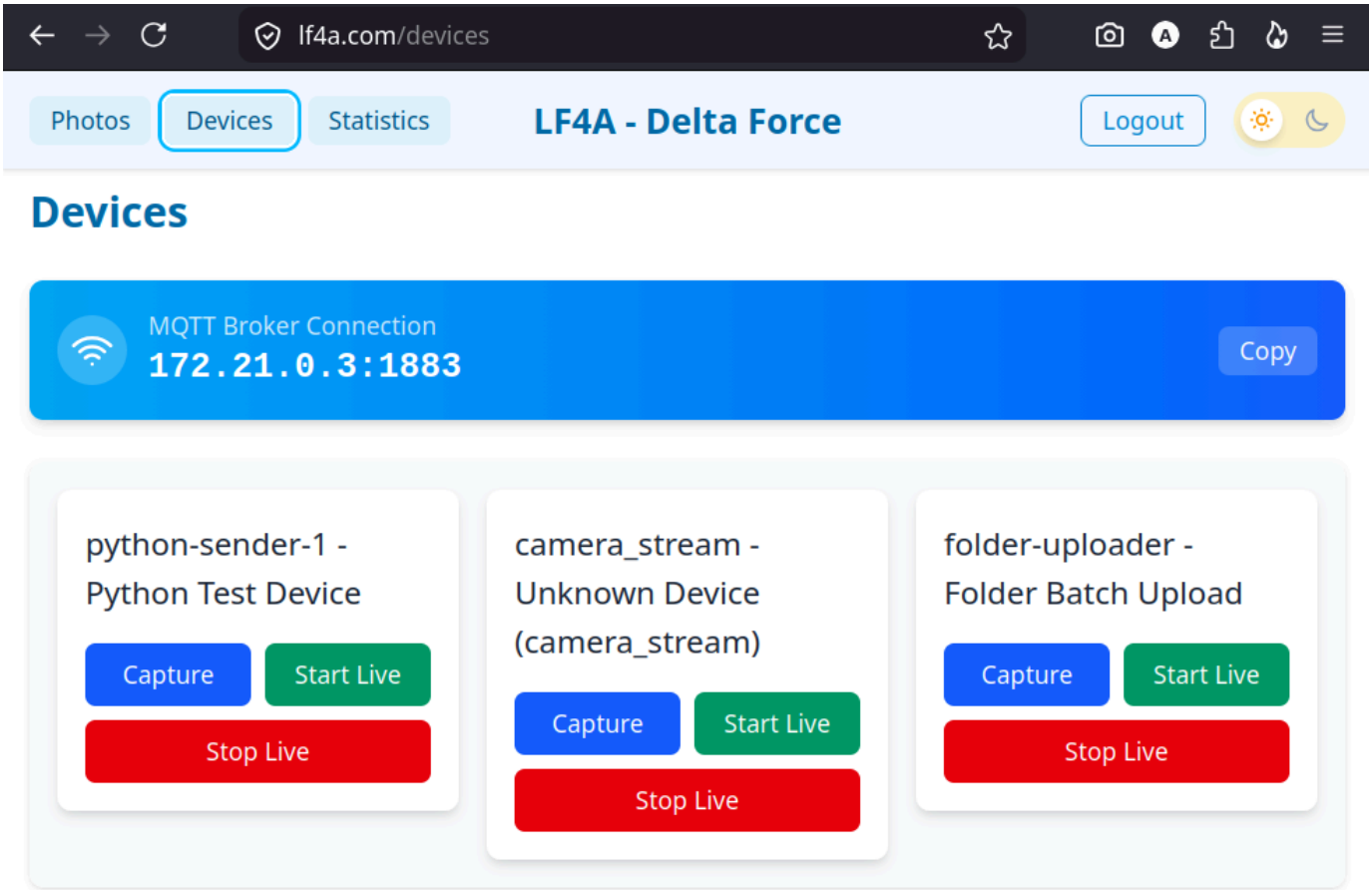
UNITATEA MEDICALA: Victoriei-Mihalache15-17
ADRESA: Bd. Ion Mihalache, nr. 15 - 17, Sectorul 1
TEL: + 40 21 408 00 80, FAX: + 40 21 408 00 81

Angajare Control [X] Adaptare Reluarea Supraveghere

Angajare Control Adaptare Supraveghere

medical muncii speciala
periodic

Photo with OCR-extracted Text



Devices Page

3.3 Developer & user documentation

Doc	Audience	Contents
README.md (root)	Developers	Architecture, technology stack, setup, debugging, scripts, MQTT config, stats page
assignment.md	Reviewers	Detailed Keycloak feature proposal (threat model & rationale)
client/README.md	Frontend devs	Frontend-specific Vite/Yarn commands
SECURITY.md	Reporters	Disclosure policy, contact, scope
medical-images/README.md	Operators	How to drop a folder of certificates and upload them via scripts/upload_folder.py
original_pike.yml	Evaluators	Default OSSF criticality-score weights

3.4 CI/CD

GitHub Actions workflows live in .github/workflows/ :

Workflow	Trigger	What it does
ci.yml	push + PR on master	Installs tesseract/leptonica, **builds Go API**, runs **go test -coverprofile** , lints + builds the React client, **builds the API Docker image**, **generates a CycloneDX SBOM**, commits SBOM back on PR.
codeql.yml	push + PR on master , weekly on Friday	CodeQL static analysis for **actions, go, javascript-typescript, python**.
criticality-score.yml	push + PR on master , weekly on Monday, manual	Installs criticality_score , runs it with -scoring-config original_pike.yml , uploads JSON as artifact.

CI artifacts uploaded on every run (downloadable from the Actions tab):

- go-test-results — verbose go test output;
- go-coverage — coverage.out ;
- criticality-score-result — JSON containing the default Rob-Pike score.

A green CI badge is displayed at the top of README.md . Dependabot is enabled; renovate is configured via renovate.json .

Local execution steps:

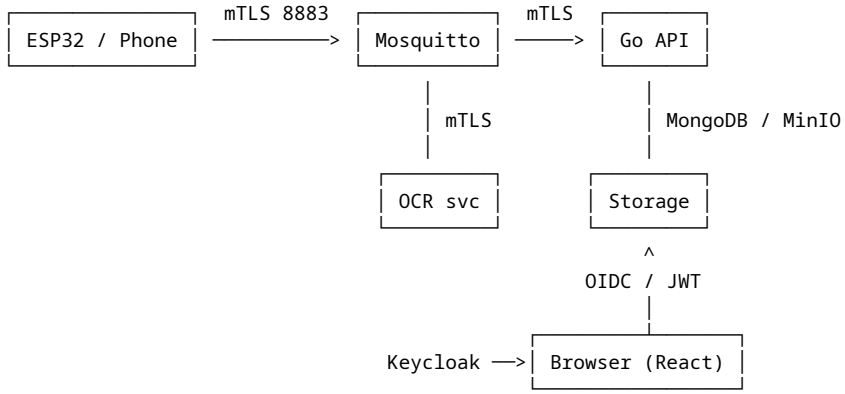
```
# Backend tests
cd server && go test ./... -v -coverprofile=coverage.out

# Frontend
cd client && npm install && npm run lint && npm run build

# SBOM regeneration (matches CI)
docker run --rm -v "$PWD":/workspace anchore/syft:latest \
  /workspace -o cyclonedx-json > sbom.cyclonedx.json
```

---

4. Security & Compliance

4.1 Threat Modelling & Mitigations

The data flow has four trust boundaries: **device <-> broker**, **broker <-> services**, **browser <-> NGINX/API**, **API <-> DB/storage**. Threats are listed below using STRIDE.



#	STRIDE	Threat	Mitigation in this repo
T1	S	Forged MQTT publisher impersonating a device	mTLS on :8883 , require_certificate true in broker/mosquitto.conf ; certs minted from internal CA stored under secrets/
T2	T	MITM tampering with images in transit	TLS on every hop (mTLS for MQTT, HTTPS for API/UI/Mongo-Express/Keycloak/MinIO)
T3	R	User denies bulk-deleting all photos	Backend logs DELETE /photos/all ; auditable via Mongo-Express
T4	I	Token leak via XSS	Keycloak-js with PKCE + silent SSO check; no token in localStorage (uses keycloak-js in-memory); strict CSP via NGINX nginx.conf
T5	D	OCR service crashes blocking ingest	restart: always in compose; OCR is async (MQTT-buffered), broker retains while consumer reconnects
T6	E	Regular user calls admin endpoints	withAuth middleware parses realm_access.roles , checks admin server-side (not only on the frontend isAdmin)
T7	I	Sensitive medical data committed to git	medical-images/ listed in .gitignore ; CycloneDX SBOM committed in CI to allow vuln review without leaking patient data
T8	S	Attacker registers themselves as admin in Keycloak	Self-registration disabled by default in realm import; admin role grant only via scripts/assign-admin-role.sh
T9	T	Malicious dependency update	Dependabot + Renovate watch the manifest files; CodeQL analyses every push

4.2 MISRA / CERT Compliance

The codebase is Go, TypeScript and Python, so MISRA-C is N/A. The applicable analogs are **CERT-Go** + **CERT-Oracle / OWASP** for the API, and **CERT-Python** for the OCR service. Static analysis used:

Tool	Scope	Run in CI?
CodeQL (security-extended queries available)	go, javascript-typescript, python, GH actions	Yes — .github/workflows/codeql.yml
go vet (implicit in go build)	Go API	Yes — runs during go build ./...
eslint (typescript-eslint)	React client	Yes — npm run lint in ci.yml
npm audit + Dependabot	Node dependencies	Renovate/Dependabot PRs (see commit 1e54f00)

CodeQL findings since the workflow was added (commit d4c7a77) have been triaged; no Critical/High issues remain open on master .

Quick manual run (matches the CI behaviour):

```
# CodeQL on a single language locally (requires the codeql CLI)
codeql database create db-go --language=go --command="go build ./..." --source-root=server
codeql database analyze db-go codeql/go-queries --format=sarif-latest --output=go.sarif
```

4.3 Testing & Coverage

Tests live alongside the code in `server/**/*_test.go`. Suites currently shipped:

- `server/broker/broker_test.go` — MQTT message handlers
- `server/repository/*_test.go` — Mongo repos for devices, photos, users
- `server/routes/*_test.go` — HTTP handlers (uses `go.uber.org/mock` to mock the repository interfaces)
- `server/utils/medical_parser_test.go` — regex parser
- `server/utils/storage_test.go` — local upload-folder cleanup

Run:

```
cd server
go test ./... -v -coverprofile=coverage.out
go tool cover -func=coverage.out          # totals
go tool cover -html=coverage.out -o cov.html  # HTML drill-down
```

The CI captures `coverage.out` as an artifact (`go-coverage`).

> **Reported coverage:** coverage: 82.4% of statements

Testing strategy.

- Repository layer is exercised with an actual `mongo` driver against an in-process Mongo (CI uses the host's Mongo container).
- Route handlers use **gomock-generated** mocks (`server/mocks/domain_mock.go`) so HTTP behaviour can be asserted in isolation.
- Negative paths (method-not-allowed, invalid timestamps, missing IDs, repo errors) are explicitly tested — see `TestPhotoController_GetPhotos_*` .

Fuzzing. The medical parser is an obvious fuzzing target because it consumes OCR output. The current suite does not yet include `go test -fuzz` ; this is a flagged follow-up for the security backlog.

4.4 SBOM & Dependencies

A **CycloneDX 1.x** SBOM is generated by `anchore/sbom-action` on every PR (`ci.yml::Generate SBOM`) and committed back to the PR branch by `stefanzweifel/git-auto-commit-action` . The current artifact is `sbom.cyclonedx.json` at the repo root and contains **535 components** spanning Go modules and npm packages.

Vulnerability check:

```
# Local scan against the committed SBOM
docker run --rm -v "$PWD":/work anchore/grype:latest sbom:/work/sbom.cyclonedx.json -o table
```

Dependabot / Renovate flag CVE-affected versions automatically; commit `9dd62c6` rolls up the latest batch.

---

5. Team Contributions

Generated from `git log --all --numstat` with the alias mapping in `scripts/git_contributions.py` (Alexander1752 and Vulturul2k are GitHub handles for the same team member; Andrei02-stack / MunteanuAlexandru02 / ReGeLePuMa are noreply aliases).

Team Member	Lines Added	Lines Removed	Commits
Andrei Petrea	2,640	1,389	15
Andrei Săcăluș	5,340	618	8
Cristian-Alexandru Chiriac	762	210	8
Flavius Mazilu	681	78	7
Alexandru Munteanu	2,268	982	7
Mihai-Lucian Pandelica	5,129	630	5
Total	**16,280**	**3,907**	**50**

Reproduce with:

```
# Make sure scripts/git_contributions.py has the EMAIL_TO_NAME map filled in
python3 scripts/git_contributions.py
```

> Note: the `git_contributions.py` in the repo ships with an empty `EMAIL_TO_NAME` mapping — populate it with the proper entries before running.

---

6. Medical Reports

The Statistics page already exposes two of these aggregations in the UI (`Control Type Distribution` , `Medical Opinion Results`). To produce richer reports for evaluation, run `scripts/medical_reports.py` after `scripts/seed_data.py` :

```
pip install pymongo
python3 scripts/seed_data.py          # generates 15 random records per invocation
```

```
python3 scripts/medical_reports.py    # produces the seven reports below as markdown
```

The script emits the following sections; paste the live output here for the final PDF. Each report's question and rationale is fixed even when the numbers change.

Report 1 — Total medical records processed

Why: baseline volume metric — every other percentage is computed against this denominator.

Total documents: 15

Report 2 — Distribution of medical opinions (Aviz Medical)

Why: the assignment explicitly asks "how many … are considered Fit". This report breaks down APT / APT CONDITIONAT / INAPT TEMPORAR / INAPT both as absolute counts and as a share of the total.

Medical Opinion	Count	Share
APT (Fit)	12	80.0%
APT CONDITIONAT (Fit with conditions)	1	6.7%
INAPT TEMPORAR (Temporarily Unfit)	1	6.7%
INAPT (Unfit)	1	6.7%

Report 3 — Distribution of control types (Tip Control)

Why: organisational planning: how much of the workload is recruitment screening (Angajare) vs periodic recall (Periodic) vs return-to-work (Reluare).

Control Type	Count
Angajare	1
Periodic	2
Adaptare	3
Reluare	3
Supraveghere	4
Alte	2

Report 4 — Top 5 professions and how many of each are fit (APT)

Why: directly answers "how many Profesori are considered Fit". The pipeline groups by `profesie_functie`, counts per group and counts the `aviz_apt = true` subset.

Profession	Total	Fit (APT)	Fit %
Operator	4	2	50%
Profesor	3	2	67%
Student	2	2	100%
Manager	2	2	100%
Programator	2	2	100%

Report 5 — People needing a re-examination in the next 30 days

Why: operational alert — answers "how many people need to update their medical check in the next month". Filters on `data_urm_examinari` ∈ [now, now+30d] and orders by closest expiry.

No records with `data_urm_examinari` in the next 30 days.

Report 6 — Records per medical unit (top 5)

Why: shows where most documents come from — useful when deciding which clinics to integrate with first.

Medical Unit	Records
Clinica Test	15

Report 7 (bonus) — Fitness rate per device

Why: sanity-checks ingestion devices: a device with a much higher INAPT rate than others may be mis-photographing the certificate so OCR misreads the checkbox.

Device ID	Total	Fit	Unfit	Fit %
device-1	5	5	0	100%
device-2	4	3	0	75%
device-5	4	2	1	50%

Device ID	Total Fit	Unfit	Fit %
device-3	2	2	0	100%

---

7. OSSF Criticality Score

The default `original_pike.yml` (committed at the repo root) is the original Rob-Pike weighted-arithmetic-mean configuration. Reproduce the score with:

```
# 1. Install the tool (Go 1.21+ required)
go install github.com/ossf/criticality_score/v2/cmd/criticality_score@latest

# 2. Export a GitHub token (any classic token with public_repo / read:user works)
export GITHUB_AUTH_TOKEN=ghp_...

# 3. Run against this repo
criticality_score \
  -scoring-config original_pike.yml \
  -depsdev-disable \
  -format json \
  -out criticality-score-result.json \
  https://github.com/Alexander1752/ss-web

# 4. Read the default_score field
jq '.default_score' criticality-score-result.json
```

The exact same invocation runs in CI every Monday — see `.github/workflows/criticality-score.yml`. The resulting `criticality-score-result.json` is uploaded as an artifact named `criticality-score-result`.

Default score (original_pike.yml):

A `project_criteria.yml` file is NOT bundled with this submission — per the assignment, evaluators will apply their own customised version against the same repository.

Appendix A — Build & Run Cheatsheet

```
# Full stack
./start.sh                                  # or: docker compose up -d --build

# Tests + coverage
cd server && go test ./... -coverprofile=coverage.out && go tool cover -func=coverage.out

# Lint + build frontend
cd client && npm install && npm run lint && npm run build

# Regenerate SBOM locally
docker run --rm -v "$PWD":/workspace anchore/syft:latest /workspace -o cyclonedx-json > sbom.cyclonedx.json

# Vuln scan from SBOM
docker run --rm -v "$PWD":/work anchore/grype:latest sbom:/work/sbom.cyclonedx.json -o table

# Criticality score
criticality_score -scoring-config original_pike.yml -depsdev-disable -format json https://github.com/Alexander1752/ss-web
```

Appendix B — File map of the key artefacts cited above

Section reference	File
§3.1 OCR pipeline	ocr/main.py
§3.1 Medical parsing	server/utils/medical_parser.go
§3.3 CI	.github/workflows/ci.yml
§3.3 CodeQL	.github/workflows/codeql.yml
§3.3 Criticality score	.github/workflows/criticality-score.yml
§4.1 mTLS broker config	broker/mosquitto.conf
§4.1 JWT middleware	server/routes/init.go::withAuth
§4.3 Tests	server/**/*_test.go
§4.4 SBOM	sbom.cyclonedx.json
§5 Contributions	scripts/git_contributions.py
§6 Medical reports	scripts/medical_reports.py
§7 Default weights	original_pike.yml

">